

COMSATS University Islamabad
Abbottabad, Pakistan

AGRI APP

By

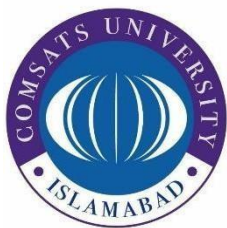
HashimMuneeb	CIIT/FA20-BCS-136/ATD
Waseem Tasawar	CIIT/FA20-BCS-133/ATD
Abdul Wahab Malik	CIIT/FA20-BCS-114/ATD

Supervisor

Sumair Khan

Bachelor of Science in Computer Science (2020-2024)

The candidate confirms that the work submitted is their own and appropriate credit has been given where reference has been made to the work of others



COMSATS University, Islamabad Pakistan

AGRI APP.

**A project presented to
COMSATS Institute of Information Technology, Islamabad**

**In partial fulfillment
of the requirement for the degree of**

Bachelor of Science in Computer Science (2020-2024)

By

HashimMuneeb	CIIT/FA20-BCS-136/ATD
Waseem Tasawar	CIIT/FA20-BCS-133/ATD
Abdul Wahab Malik	CIIT/FA20-BCS-114/ATD

DECLARATION

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Hashim Muneeb

Waseem Tasawar

Abdul Wahab Malik

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS (CS) “AGRI APP” was developed by **Hashim Muneeb (CIIT/FA20-BCS-136), Waseem Tasawar (CIIT/FA20-BCS-133)** and **Abdul Wahab Malik (CIIT/FA20-BCS-114)** under the supervision of “SUMAIR KHAN” and that in his opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Computer Sciences.

Supervisor

External Examiner

Head of Department
(Department of Computer Science)

EXECUTIVE SUMMARY

The Agri App project embodies a structured approach to enhance agricultural practices in Pakistan through a robust digital solution. It comprises a suite of modules meticulously designed to cater to the diverse needs of farmers nationwide.

Central to the Agri App are modules such as Farmer Management, Equipment Rental, Real-Time Monitoring (including Water and Fertilizer Needs), Crop Information, and Loans and Resources. These modules operate cohesively, leveraging React Native's functional components and methods to ensure modularity and reusability. This structural approach facilitates the seamless integration of new functionalities and enhancements across interconnected modules.

The project's methodology emphasizes agility, adopting iterative development cycles and continuous feedback loops. This agile framework enables rapid adaptation to evolving user needs, ensuring that the Agri App remains responsive and aligned with the dynamic requirements of the agricultural community.

Developed using React Native, the Agri App provides an intuitive user experience on Android platforms, supporting a wide array of devices and operating systems. Farmers benefit from straightforward access to essential tools and information, optimizing agricultural operations and enabling informed decision-making.

With its modular architecture, the Agri App fosters collaboration and operational efficiency within the agricultural ecosystem. For example, the Farmer Management Module acts as a comprehensive dashboard, integrating features like real-time monitoring, equipment rental services, and crop-specific information dissemination. Simultaneously, the Equipment Rental Module facilitates easy access to farming tools, while Real-Time Monitoring Modules offer critical sensor-based insights for informed decision-making.

In conclusion, the Agri App signifies a significant stride in modernizing Pakistan's agricultural sector. By harnessing cutting-edge technology and a structured development approach, the Agri App aims to empower farmers, boost productivity, and promote sustainable agricultural practices nationwide.

ACKNOWLEDGEMENT

All praise is to Almighty Allah who bestowed upon us a minute portion of His boundless knowledge by virtue of which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor “Sumair Khan”. Without their personal supervision, advice, and valuable guidance, completion of this project would have been doubtful. We are deeply indebted to them for their encouragement and continual help during this work.

And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us the values of honesty & hard work.

Hashim Muneeb

Waseem Tasawar

Abdul Wahab Malik

ABBREVIATIONS

SRS	Software Require Specification
PC	Personal Computer
SDA	Software Design and Architecture
DFD	Data Flow Diagram
AI	Artificial Intelligence
SDLC	Software Development Life Cycle
HCI	Human Computer Interaction
UI	User Interface
SPM	Software Project Management
SEC	Software Engineering Concepts
SRE	Software Requirement Engineering
SQE	Software Quality Engineering
HTML	Hyper Text Markup Language
CSS	Cascading Style Sheet
JS	Java Script
NST	Neural Style Transfer

TABLE OF CONTENTS

1	Introduction.....	01
1.1	Brief	01
1.2	Relevance to Course Modules	01
1.3	Project Background	01
1.4	Literature Review	01
1.5	Analysis from Literature Review (in the context of your project)	02
1.6	Methodology and Software Lifecycle for This Project	02
1.6.1	Rationale behind Selected Methodology	02
2	Problem Definition.....	03
2.1	Problem Statement	03
2.2	Deliverables and Development Requirements	03
3	Requirement Analysis.....	04
3.1	Use Cases Diagram(s)	04
3.2	Detailed Use Case	05
3.3	Functional Requirements	12
3.4	Non-Functional Requirements	15
4	Design and Architecture.....	16
4.1	Block Diagram	16
4.2	Activity Diagram	17
4.3	DFD Diagram	18
4.3	Sequence Diagram	19
5	Implementation.....	25
5.1	Algorithm	25
5.2	User Interface	26
6	Testing and Evaluation.....	38
6.1	Manual Testing	38
6.1.1	System testing	38
6.1.2	Unit Testing	38
6.1.3	Functional Testing	45
6.1.4	Integration Testing	49
7	Conclusion and Future Work.....	50
7.1	Conclusion	50
7.2	Future Work	51
8	Reference.....	53

LIST OF FIGURES

Fig 3.1 Use Case Diagram.....	14
Fig 4.1 Block Diagram	27
Fig 4.2 Activity Diagram	28
Fig 4.3 Data Flow Diagram.....	29
Fig 4.4 Sequence Diagram for Login.....	30
Fig 4.5 Sequence Diagram for Signup	31
Fig 4.6 Sequence Diagram for Water Need.....	32
Fig 4.7 Sequence Diagram for Rent Machine	33
Fig 4.8 Sequence Diagram for Real-Time Monitoring.....	34
Fig 4.9 Sequence Diagram for Add New Machine	35
Fig 5.1 Home Screens	37
Fig 5.2 Login Screens	38
Fig 5.3 Signup Screen	39
Fig 5.4 Farmer Dashboard and Profile Screen	40
Fig 5.5 Moisture and Real-Time Screen	41
Fig 5.6 NPK and values Screen	42
Fig 5.7 View and rent machines Screen.....	43
Fig 5.8 See Booking and Rent Screen.....	44
Fig 5.9 Loans and Insurances Screen.....	45
Fig 5.10 MachineOwner Dashboard and Profile Screen.....	46
Fig 5.11 Add, Delete and Update Screen.....	43
Fig 5.12 Machine Reviews and Booking Screen	44

LIST OF TABLES

Table 2.1 Deliverables and Development Requirements	13
Table 3.1 UC Login/Signup	15
Table 3.2 UC Equipment Rental	16
Table 3.3 UC Real-Time Land Monitoring	17
Table 3.4 UC Water Needs.....	18
Table 3.5 UC Fertilizer Needs	19
Table 3.6 UC Crop Information	20
Table 3.7 UC News and Resources	21
Table 3.8 FR Equipment Rental.....	22
Table 3.9 FR Real-Time Data Monitoring	23
Table 3.10 FR Water Need.....	23
Table 3.11 FR Fertilizer Needs	24
Table 3.12 FR Loans and Resources Section	24
Table 3.13 FR Dashboard Login/Signup Integration.....	25
Table 5.1 Open Weather API	36
Table 6.1 Unit Testing for Registration	49
Table 6.2 Testing 2 Add Machinery	49
Table 6.3 Testing 3 Fetch Machinery Orders	50
Table 6.4 Testing 4 Delete Machinery	51
Table 6.5 Testing 5 support chat	52
Table 6.6 Testing 6 Machine Owner logout.....	53
Table 6.7 Testing 7 <i>Sensor Data Integration</i>	54
Table 6.8 Testing 8 display temperature and humidity	55
Table 6.9 Testing 9 machine owner login	56
Table 6.10 Testing 10 Machine Rental Process.....	57
Table 6.11 Testing 11review process	58
Table 6.12 Testing 12 previous booking.....	59
Table 6.13 Testing 13 NPK sensor handling.....	60

1. Introduction

In the contemporary agricultural landscape of Pakistan, there exists an urgent need for innovative solutions to address the myriad challenges faced by farmers and propel the modernization of farming practices. The Agri App project emerges as a timely response to this pressing demand, aiming to revolutionize agricultural operations through the implementation of a comprehensive digital platform. By leveraging cutting-edge technology, the Agri App seeks to empower farmers with tools and resources that enhance productivity, streamline decision-making processes, and promote sustainable agricultural practices.

1.1. Brief

The Agri App project endeavors to provide farmers in Pakistan with a user-friendly digital platform offering essential tools and resources to augment their productivity and decision-making capabilities. By consolidating various functionalities within a single application, the Agri App aims to streamline agricultural operations, facilitate access to crucial information, and empower farmers to optimize their farming practices effectively.

1.2. Relevance to Course Modules

The Agri App project, a cornerstone of the BCS program, draws upon a diverse range of course modules to create a transformative digital solution for agriculture. By leveraging principles from data structures, database management, software engineering, mobile development, microprocessors and assembly language, the Agri App addresses complex agricultural challenges. These modules synergize to enhance transparency, security, and efficiency in agricultural information management, paving the way for sustainable agricultural practices.

1.3. Project Background

The genesis of the Agri App project stems from a comprehensive analysis of the inefficiencies and limitations inherent in traditional farming practices prevalent in Pakistan. Through extensive research and stakeholder consultations, it became evident that leveraging digital technology could offer transformative solutions to address these challenges. By harnessing the power of real-time data monitoring, predictive analytics, and user-centric design principles, the Agri App aims to revolutionize agricultural operations, thereby enhancing productivity, sustainability, and profitability for farmers across the country.

1.4. Literature Review

A thorough literature review was conducted to explore existing digital solutions in the agricultural domain, as well as insights into software development methodologies and best practices. The analysis of relevant literature highlighted the importance of modular design, real-time data monitoring, and user-centered development approaches in creating effective agricultural applications. Drawing upon insights

from academia and industry, the Agri App project seeks to incorporate these best practices to deliver a robust and user-friendly digital platform for farmers in Pakistan.

1.5. Analysis from Literature Review

The literature review underscored the significance of modular program structures in facilitating maintainability and scalability, aligning with the design approach adopted for the Agri App project. Furthermore, insights gleaned from existing research emphasized the pivotal role of real-time data monitoring and user feedback in driving continuous improvement and user satisfaction. By incorporating these insights into the development process, the Agri App aims to deliver a highly customizable and responsive platform that meets the evolving needs of farmers in Pakistan.

1.6. Methodology and Software Lifecycle for This Project

For the methodology and software lifecycle of the Agri App project, we have devised a tailored approach to ensure efficient development and successful implementation of the digital solution. This involves the selection of appropriate design methodologies and software development processes.

1.6.1. Rationale behind Selected Methodology

Procedural Approach is a highly suitable design methodology for this project. It is a widely adopted approach in software development, particularly for projects driven by the need for simplicity, ease of maintenance and a straightforward workflow.

It has following advantages:

- **Readability and Maintainability:**

The procedural paradigm promotes a clear and linear code structure, making it easier for developers to read, understand, and maintain the codebase. This is especially beneficial for projects with limited complexity.

- **Ease Of Debugging:**

Debugging is simplified in a procedural model, as developers can trace the flow of execution step by step. This can expedite the identification and resolution of issues during the development process.

- **Quick Prototyping:**

Procedural programming is often conducive to rapid prototyping. It allows for a straightforward implementation of features, facilitating the timely creation of functional prototypes to demonstrate and validate concepts.

- **Resource Efficiency:**

In scenarios where the Mobile's functionality does not require intricate object-oriented structures, a procedural approach can be more resource efficient. It allows for a leaner codebase without the overhead of unnecessary abstractions.

2. Problem Definition

The agricultural sector in Pakistan faces various challenges, including limited access to modern technology and information, inefficient resource management practices, and inadequate distribution of farming equipment and resources. Small-scale farmers, who are vital to the industry, often lack the necessary tools to optimize their farming practices and improve yields. To address these challenges, there's a crucial need for a comprehensive digital solution. The Agri App project aims to fill this gap by providing a user-friendly platform offering real-time data monitoring, equipment rental services, crop information dissemination, and access to agricultural news and resources, ultimately revolutionizing agricultural practices in Pakistan.

2.1. Problem Statement

The primary issue addressed by the Agri App project is the inefficiencies faced by farmers in Pakistan due to limited access to information, resources, and technology. The project seeks to empower farmers by providing a comprehensive digital platform that facilitates real-time data monitoring, equipment rental, crop information dissemination, and access to agricultural news and resources.

2.2. Deliverables and Development Requirements

Table 2.1 Deliverables and Development Requirements

S	Specification	Required
1	Programming Languages	React-Native, C++(Sensors)
2	Database	Cloud Firestore Database
3	Hardware Platforms	Android Mobile Phones
4	IDE	Visual Studio, Arduino
5	Sensors	DHT11(Temperature and Humidity Sensor) Soil Moisture Sensor, NPK sensor
6	Operating System	Android

3. Requirement Analysis

3.1. Use Case Diagram

The Use Case Diagram is show in Figure 3.1

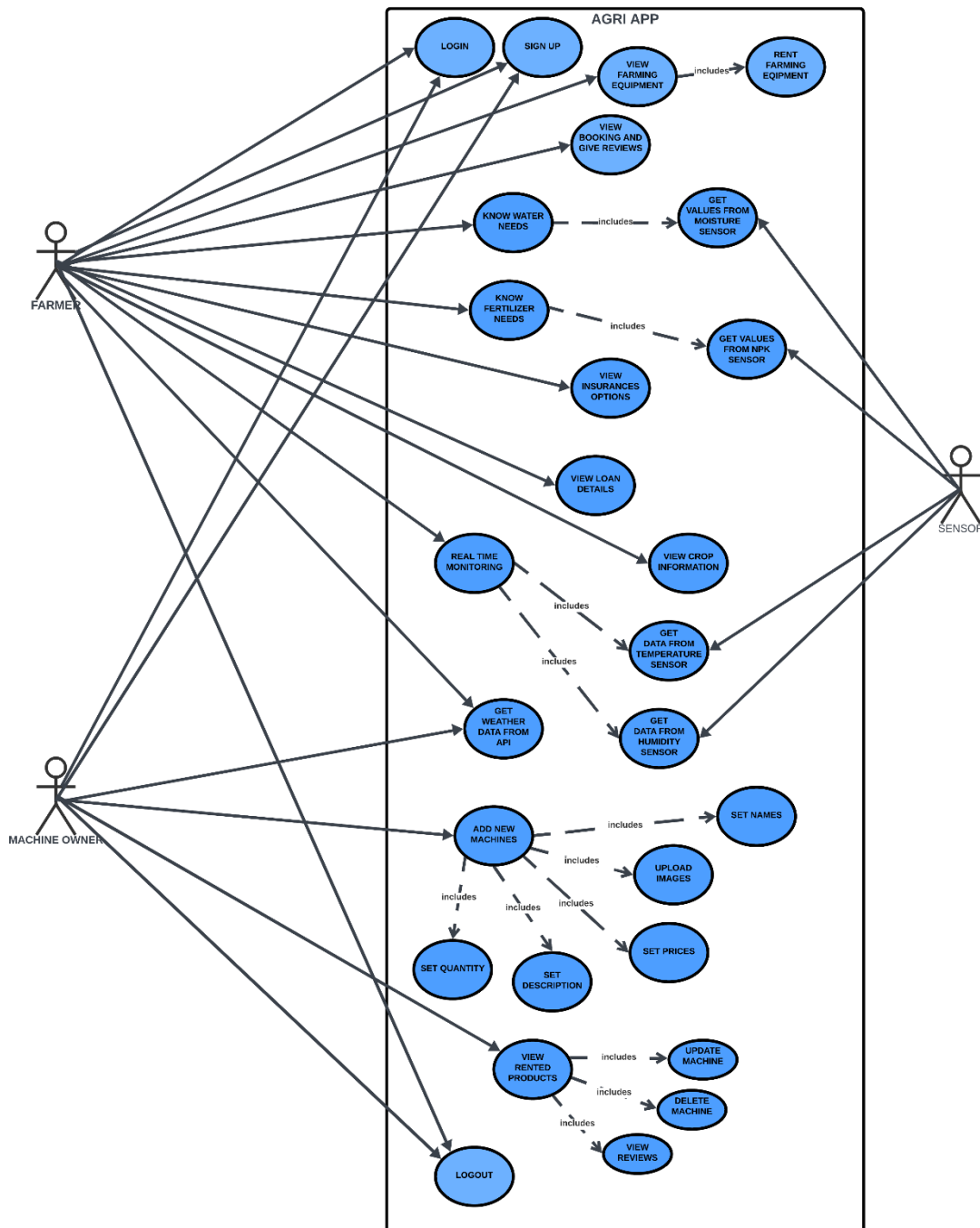


Figure 3.1 Use Case Diagram

3.2.Detailed Use Case

Table 3.2. Login/Signup

Use Case ID	UC-1
Use Case Name	Login/Signup
Actors	Farmer, Renter
Description	The Farmer and renter will signup and sign in into the application. So that they can use there own dashboards.
Trigger	The Farmer initiates an action to login/ signup to use the Farmer Module. The Renter initiates an action to login/ signup to use the Renter Module.
Preconditions	The Farmer is registered and logged into the Agriculture App.
NormalFlow	1. Farmer accesses the main dashboard. 2. Farmer navigates to the desired module. 3. Farmer interacts with the selected module. 4. Farmer completes the interaction, and relevant data is updated in the system.
Exceptions	If there is an issue with data updates or system access, notify theFarmer.
Business Rules	All functions of the Agriculture App depend on internet connectivity. Data updates are subject to system availability.

Table 3.3 Equipment Rental

Use Case ID:	UC-2
Use Case Name:	Equipment Rental
Actors:	Farmer
Description:	Farmers browse and rent farming equipment from different owners using the Equipment Rental.
Trigger:	The Farmer initiates an action to rent equipment.
Preconditions:	The Farmer is registered and logged into the Agriculture App.
NormalFlow:	1. Farmer accesses the Equipment Rental Module. 2. Farmer browses available equipment. 3. Farmer selects equipment for rent. 4. Farmer completes the rental process, and relevant data is updated in the system.
Alternative Flows:	1. Farmer requests to modify a rental. 2. Return to the relevant step in the normal flow.
Exceptions :	If the requested equipment is not available, inform the Farmer.
BusinessRues	All functions of the Agriculture App depend on internet connectivity. Equipment availability is subject to updates from EquipmentOwners.

Table 3.4 Real-Time Land Monitoring

Use Case ID:	UC-3
Use Case Name:	Real-Time Land Monitoring
Actors:	Farmer
Description:	Farmers access the Real-Time Land Monitoring Module to view real-time data collected from sensors, including temperature, humidity, and rainfall.
Trigger:	The Farmer initiates an action to monitor land conditions.
Preconditions:	The Farmer is registered and logged into the Agriculture App.
Postconditions:	Relevant data is updated in the Agriculture App database.
Normal Flow:	1. Farmer accesses the Real-Time Land Monitoring Module. 2. Farmer views real-time data on temperature, humidity, and rainfall. 3. Farmer interprets the data to make informed decisions about crops.
Exceptions :	If there is an issue fetching real-time data, notify the Farmer.
Business Rules	All functions of the Agriculture App depend on internet connectivity. Real-time data is subject to sensor availability and functionality.

Table 3.5 Water Need

Use Case ID:	UC-4
Use Case Name:	Water Need
Actors:	Farmer
Description:	Farmers access the Water Need Module to monitor soil moisture content and determine water needs for specific crops.
Trigger:	The Farmer initiates an action to check water needs.
Preconditions:	The Farmer is registered and logged into the Agriculture App.
Postconditions:	Relevant data is updated in the Agriculture App database.
NormalFlow:	1. Farmer accesses the Water Need Module. 2. Farmer monitors soil moisture content. 3. Farmer interprets data to determine water needs for specific crops.
Exceptions :	If there is an issue fetching moisture data, notify the Farmer.
BusinessRules	All functions of the Agriculture App depend on internet connectivity. Moisture data is subject to sensor availability and functionality.

Table 3.6 Fertilizer Needs

UseCase ID:	UC-5
UseCase Name:	Fertilizer Needs
Actors:	Farmer
Description:	Farmers access the Fertilizer Needs Module to analyze pH level data and receive suggestions for appropriate fertilizers based on soil acidity/alkalinity.
Trigger:	The Farmer initiates an action to check fertilizer needs.
Preconditions:	The Farmer is registered and logged into the Agriculture App.
Postconditions:	Relevant data is updated in the Agriculture App database.
NormalFlow:	1. Farmer accesses the Fertilizer Needs Module. 2. Farmer analyzes pH level data. 3. Farmer receives suggestions for appropriate fertilizers.
Exceptions :	If there is an issue fetching pH level data, notify the Farmer.
BusinessRules	All functions of the Agriculture App depend on internet connectivity. pH level data is subject to sensor availability and functionality.

Table 3.7 Crop Information

Use Case ID:	UC-6
Use Case Name:	Crop Information
Actors:	Farmer
Description:	Farmers access the Crop Classes Module to obtain information about crops commonly grown in different districts or regions.
Trigger:	The Farmer initiates an action to learn about crop classes.
Preconditions:	The Farmer is registered and logged into the Agriculture App.
Postconditions:	Relevant data is updated in the Agriculture App database.
Normal Flow:	1. Farmer accesses the Crop Classes Module. 2. Farmer views information about crops commonly grown in different districts or regions.
Exceptions :	If there is an issue fetching crop class data, notify the Farmer.
Business Rules	All functions of the Agriculture App depend on internet connectivity. Crop class data is subject to availability and updates.

Table 3.8 News and Resources

UseCase ID:	UC-7
UseCase Name:	Loans and Resources
Actors:	Farmer
Description:	Farmers access the Loans Info and Resources Module to stay informed about agriculture Loans, insurance, and subsidies.
Trigger:	The Farmer initiates an action to access Loans and resources.
Preconditions:	The Farmer is registered and logged into the Agriculture App.
Postconditions:	Relevant data is updated in the Agriculture App database.
Normal Flow:	1. Farmer accesses

3.3.Functional Requirements

FR-01: Explore and rent farming equipment with details and contact information available.

FR-02: View real-time data on temperature, humidity, and rainfall for informed decision-making.

FR-03: Access information on water needs for crops based on moisture level sensor data.

FR-04: Retrieve fertilizer recommendations based on NPK level sensor data.

FR-05: Stay updated on agriculture-related information, including loans, insurance, and subsidies.

FR-06: Communicate with rental machine owners for seamless interactions.

FR-07: Seamlessly integrate with the login/signup module for personalized profiles and secure interactions.

Table 3.9 Functional requirement for Equipment Rental

Identifier	FR-01
Title	Equipment Rental
Requirement	The dashboard shall include a module for renting equipment, providing details and contact information of available machines.
Source	Stakeholder
Rationale	Facilitate access to farming equipment for farmers.
Priority	High

Table 3.10 Functional requirement for Real-Time Data Monitoring

Identifier	FR-02
Title	Real-Time Data Monitoring
Requirement	Farmers shall be able to view real-time data monitoring, including temperature, humidity, and rainfall.
Source	Stakeholder
Rationale	Enable farmers to make informed decisions.
Priority	High

Table 3.11 Functional requirement for Water Needs for Crops

Identifier	FR-03
Title	Water Needs for Crops
Requirement	The dashboard shall display information about waterneeds for crops based on moisture level sensor data.
Source	Stakeholder
Rationale	Provide insights into crop water requirements.
Priority	High

Table 3.12. Functional requirement for Fertilizer Need

Identifier	FR-02
Title	Fertilizer Need
Requirement	Farmers shall have access to fertilizer Need based on NPK level sensor data.
Source	Stakeholder
Rationale	Support farmers in optimizing crop fertilization.
Priority	Medium

Table 3.13 Functional requirement for Loans and Resources Section.

Identifier	FR-05
Title	Loans and Resources Section
Requirement	A Loans and resources section on the dashboard shall keep farmers updated on agriculture-related information.
Source	Stakeholder
Rationale	Provide a centralized information hub for farmers.
Priority	Medium

Table 3.14 Functional Requirement for Dashboard Login/Signup Integration .

Identifier	FR-08
Title	Dashboard Login/Signup Integration
Requirement	The dashboard shall be integrated with a login/signup module for personalized farmer profiles.
Source	Stakeholder
Rationale	Provide personalized user experiences.
Priority	High

3.4.Non-Functional Requirements

NFR-01 User-Friendly

The Agri App, built using React Native, prioritizes a user-friendly experience. The interface is designed to be intuitive and avoids unnecessary complexity in its user interface (UI), ensuring that users can easily navigate and utilize its features without confusion.

NFR-02 Portability

The Agri App is developed to ensure portability, specifically targeting mobile phones. Its compatibility with various mobile devices, regardless of screen size or specifications, ensures that users can access the application seamlessly on their smartphones.

NFR-03 Security

Security is a paramount concern for the Agri App. The React Native framework allows for the implementation of robust security measures to protect user data, ensuring that the application maintains a secure environment for farmers to access and manage their agricultural information.

NFR-04 Performance

The overall performance of the Agri App is optimized for efficiency. Leveraging the capabilities of React Native, the application aims to provide a smooth and responsive user experience, even in resource-constrained environments, ensuring that farmers can rely on its performance for their daily agricultural activities.

4. Design and Architecture

The following parts of Software Design Description (SDD) report should be included in this chapter.

4.1. Block Diagram

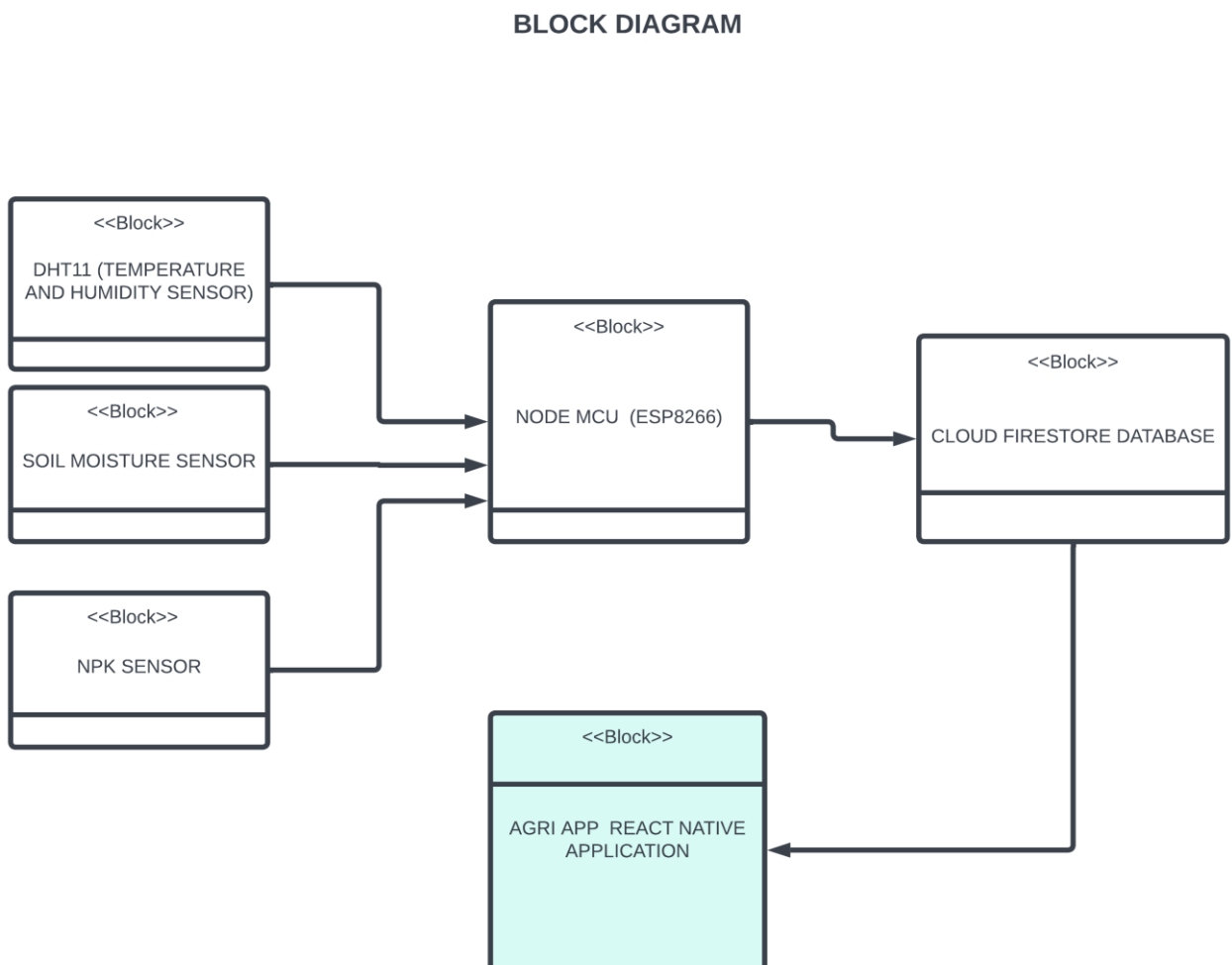


Figure 4.1. Block Diagram

17



4.3. Data Flow Diagram

4.3.1. Level 0

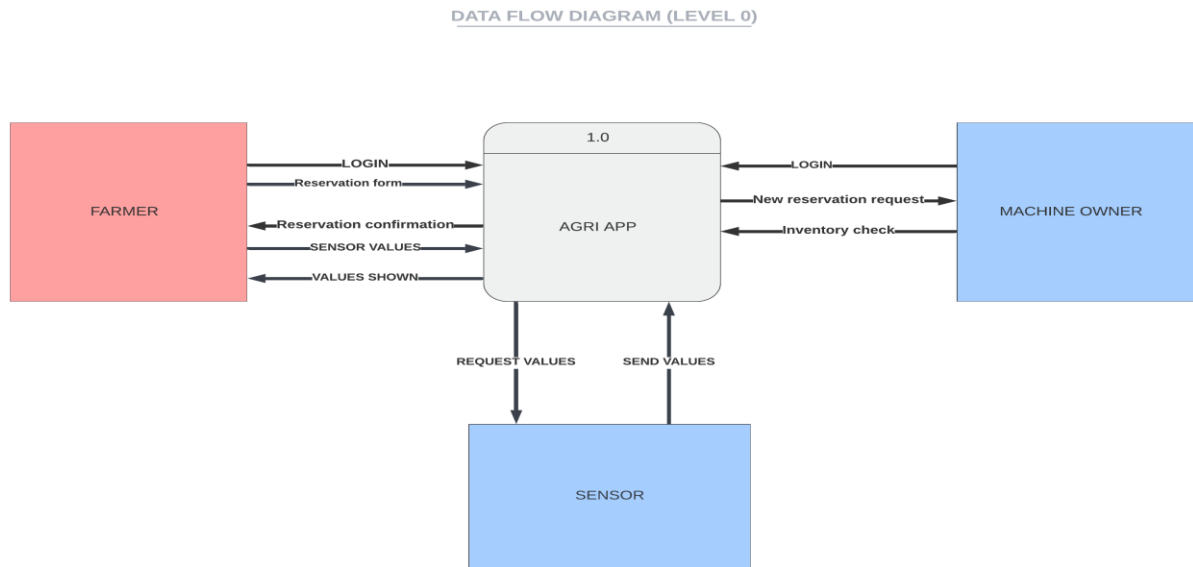


Figure 4.3 DFD Level 0

4.3.2. Level 1

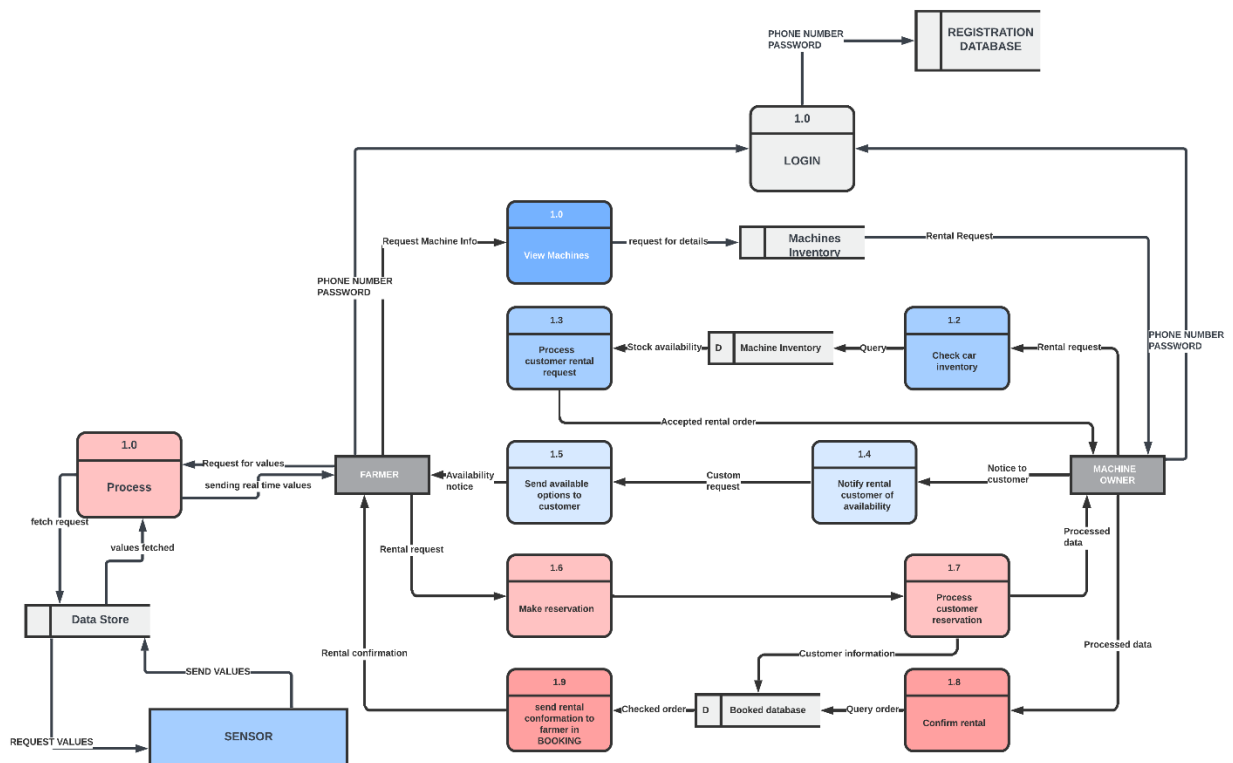


Figure 4.4 DFD Level 1

4.4. Sequence Diagram

Sequence Diagram for Login

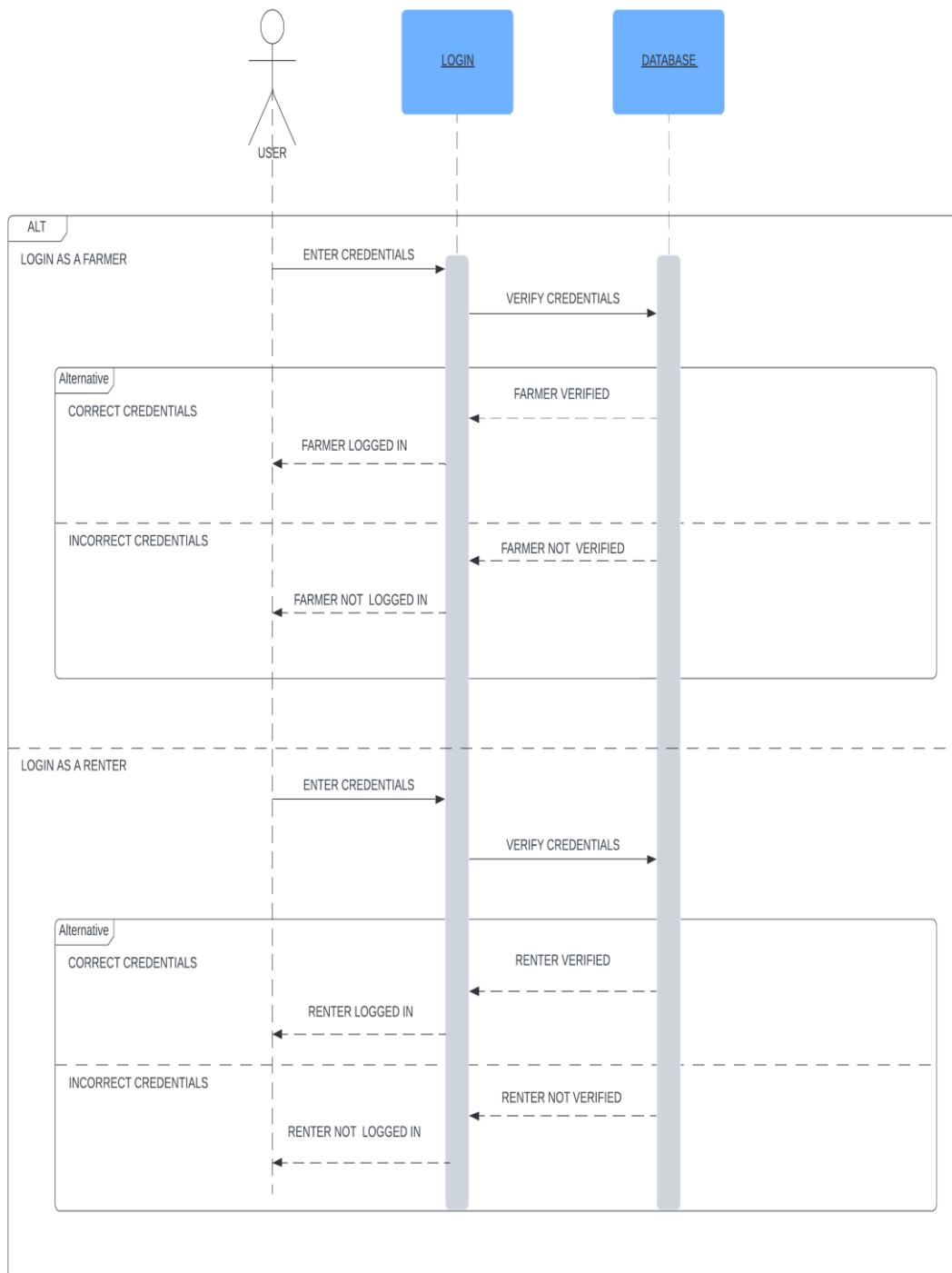


Figure 4.5 Sequence Diagram for Login

Sequence Diagram for Signup

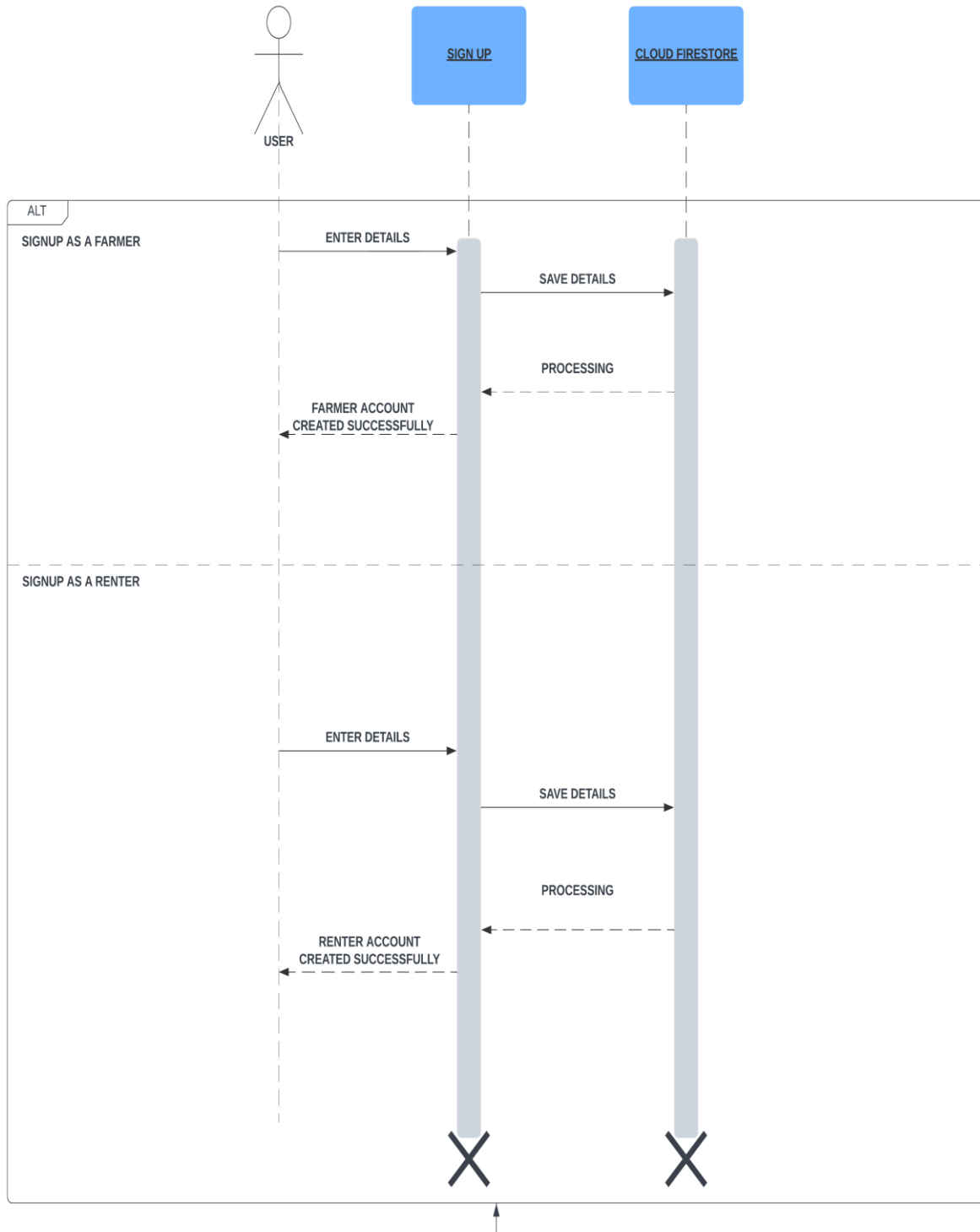


Figure 4.6 Sequence Diagram for Signup

4.4.1. Sequence Diagram for Water Need

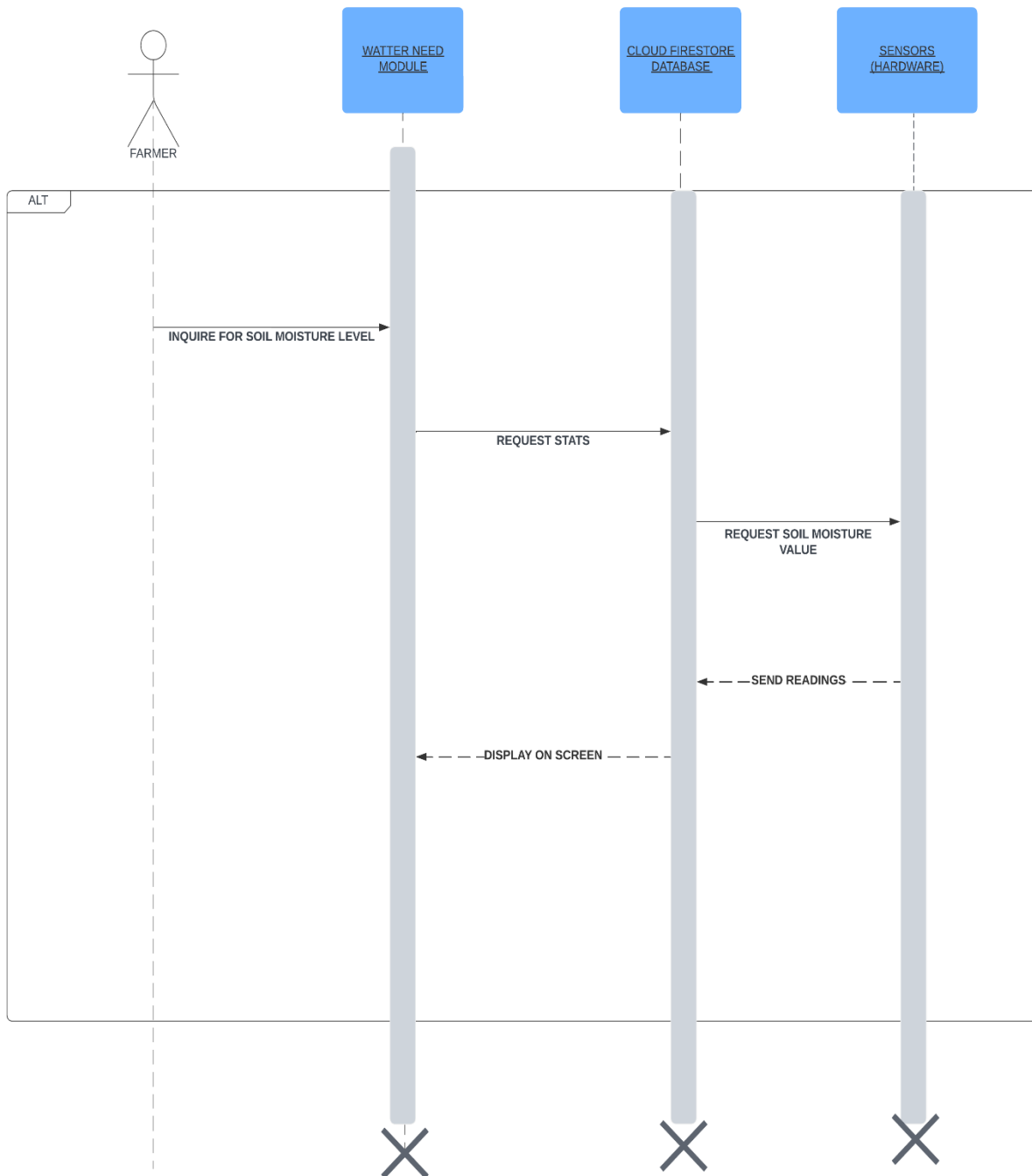


Figure 4.7 Sequence Diagram for Water Need

4.4.2. Sequence Diagram for Rent Equipment

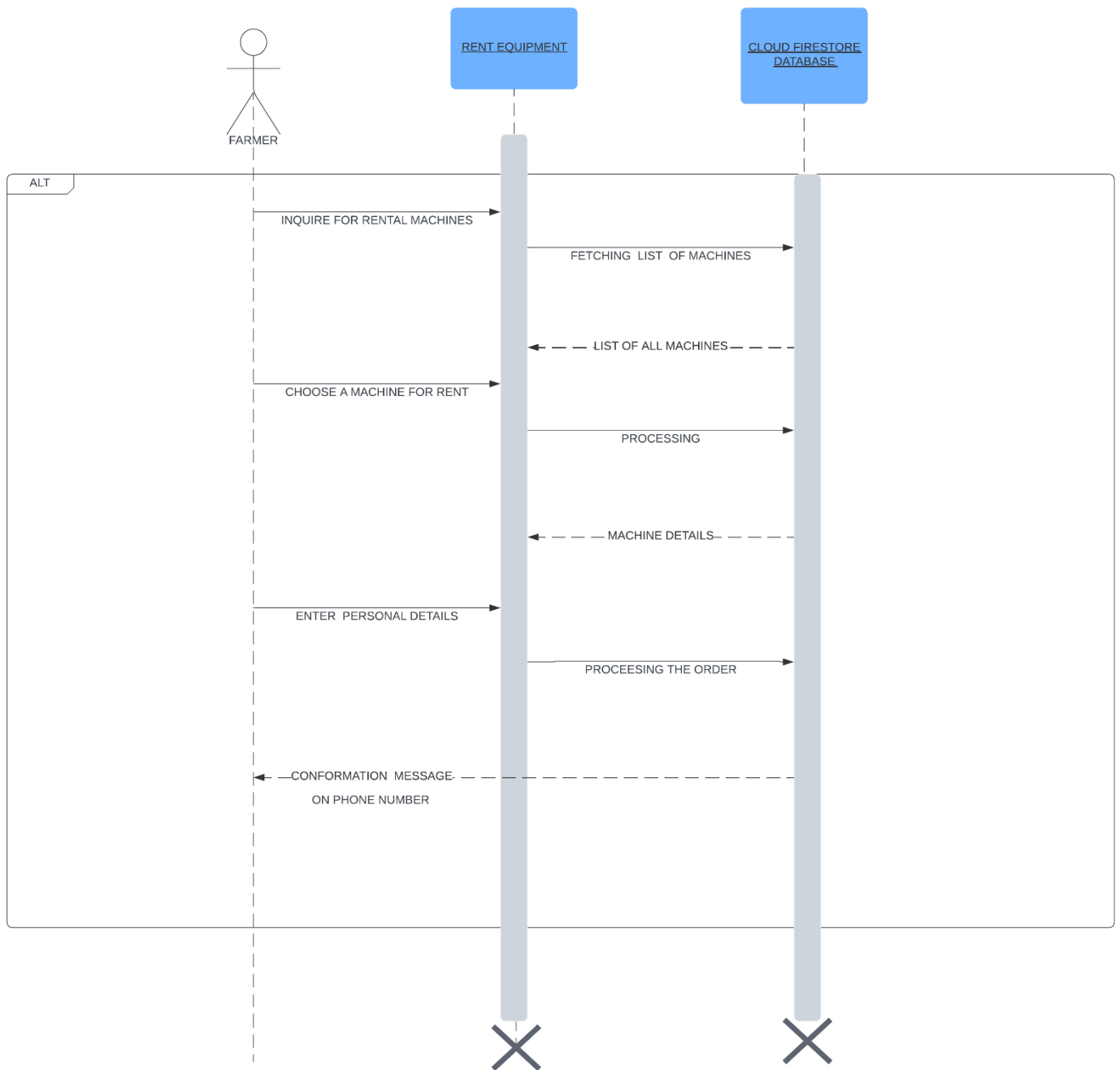
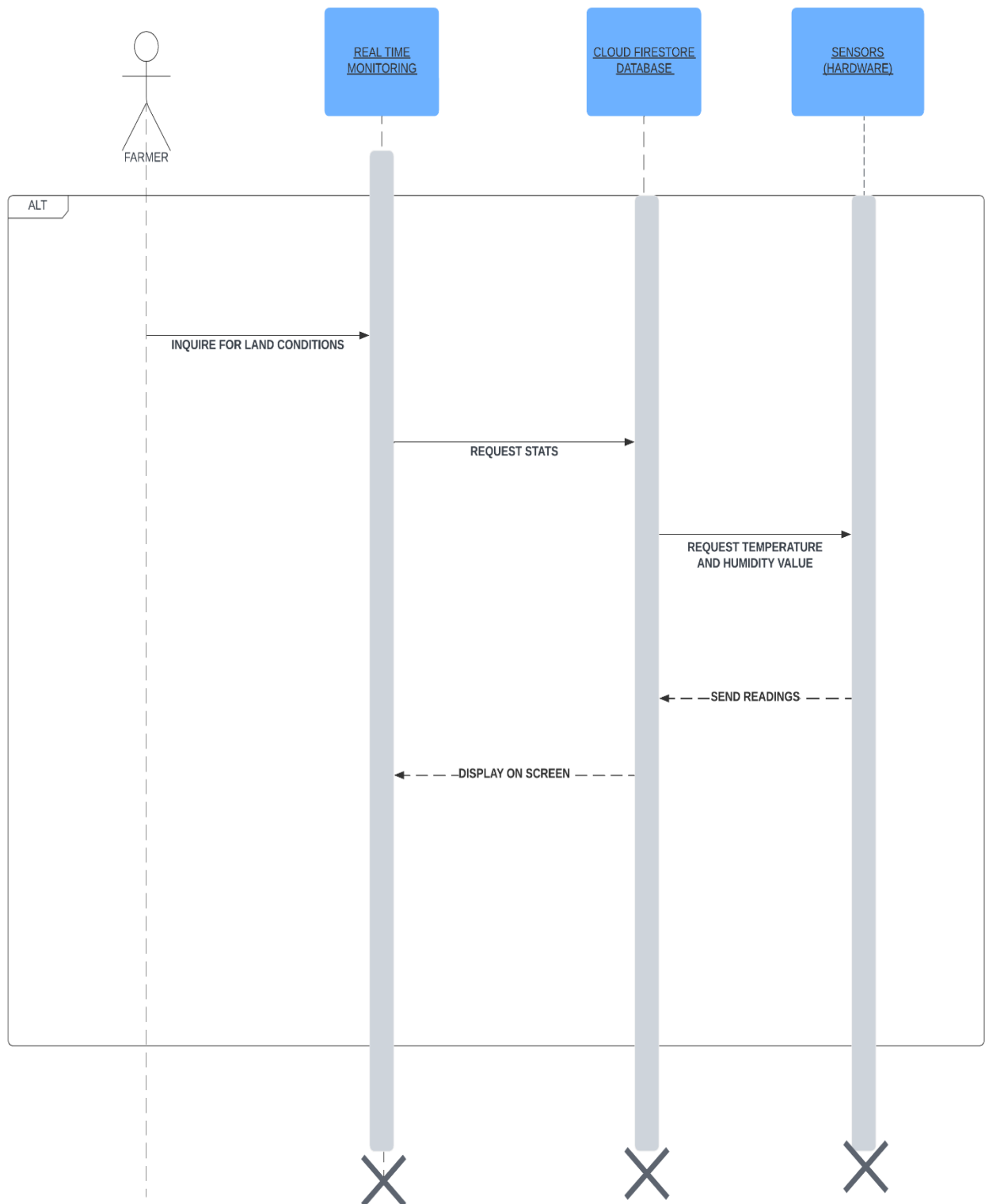


Figure 4.8 Sequence Diagram for Rent Equipment

Sequence Diagram for Real Time Monitoring*Figure 4.9 Sequence Diagram for Real Time Monitoring*

Sequence Diagram for Add New Machine

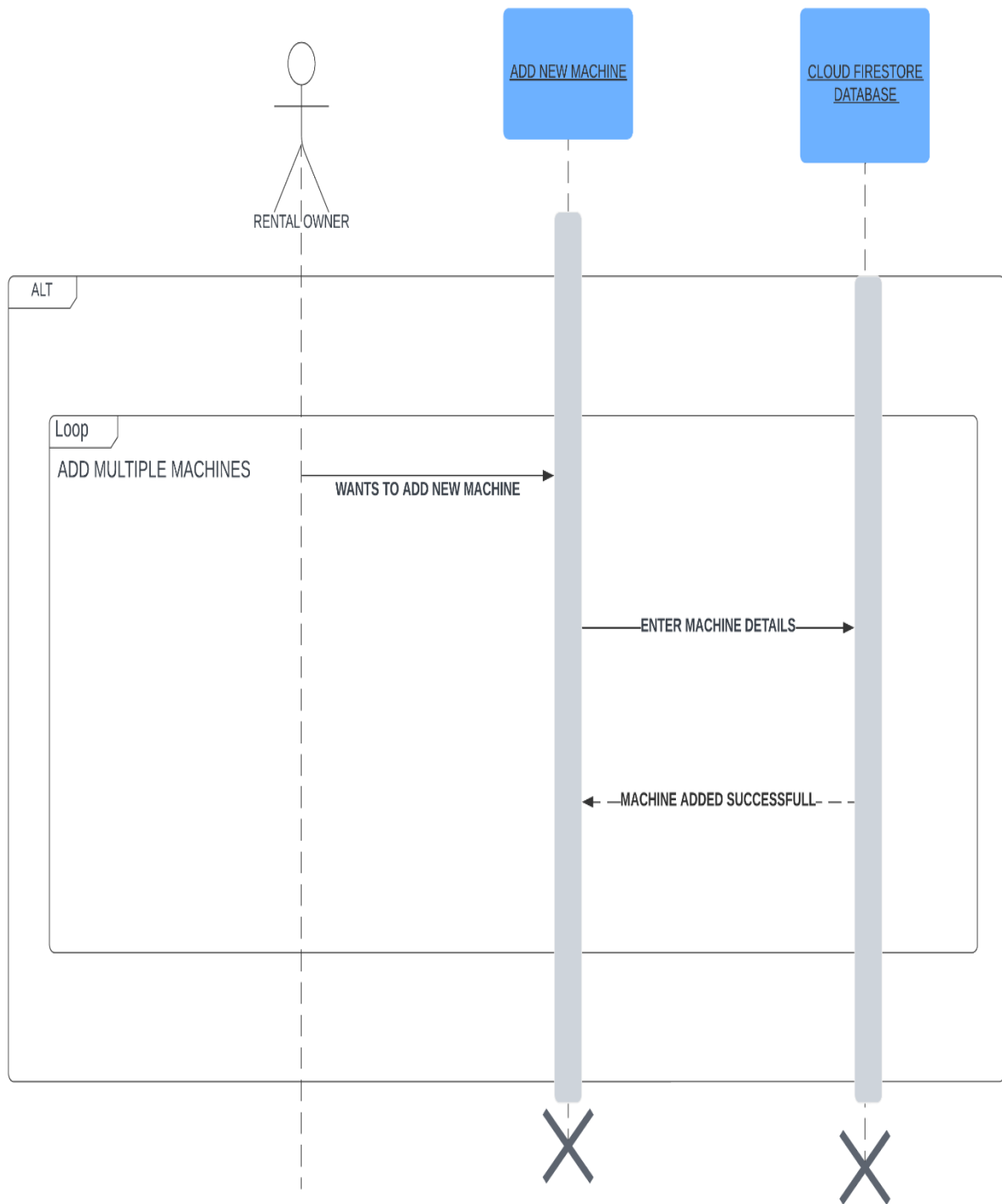


Figure 4.10 Sequence Diagram for Add New Machine

5. Implementation

In this portion, we will define the implementation of Agri App.

5.1. Algorithm

- **Machine Rental for Farmers:** Farmers can rent expensive machinery, making advanced equipment accessible without the need for large capital investment.
- **NPK Sensor for Fertilizer Needs:** The app uses NPK sensors to determine the fertilizer needs of the crops, ensuring optimal nutrient levels for healthy growth.
- **Moisture Sensor for Water Needs:** The moisture sensor provides data on soil moisture levels, helping farmers to manage irrigation efficiently and avoid water wastage.
- **Temperature and Humidity Sensors for Real-Time Monitoring:** The app includes temperature and humidity sensors to monitor environmental conditions in real-time, allowing farmers to take timely actions to protect their crops from adverse weather conditions.
- **Loan and Insurance Information:** Farmers can view information about loans and insurance options provided by different banks, helping them make informed financial decisions.
- **Chat for Help:** Farmers can use the chat feature to seek help and advice, facilitating direct communication with experts and support services.
- **Machine Management for Owners:** Machine owners can add, delete, and update the details of their machines, allowing them to manage their rental inventory effectively.

5.2. External APIs

Table 5.1: Open Weather API

Name of API	Description of API	Purpose of usage	List down the function/class name in which it is used
Open Weather API	Provides current and forecast weather data	Fetch and display weather reports	WeatherService, WeatherReportComponent in Machine Owner and Farmer Dashboard.

5.3. User Interface

Home Screens



Figure 5.1 Home Screens

Login as a Machine Owner



Login as a Farmer



Figure 5.2 Login Screens

Sign up as a Machine Owner

Signup as a Farmer



Figure 5.3 Signup Screens

Farmer Dashboard



Farmer Profile

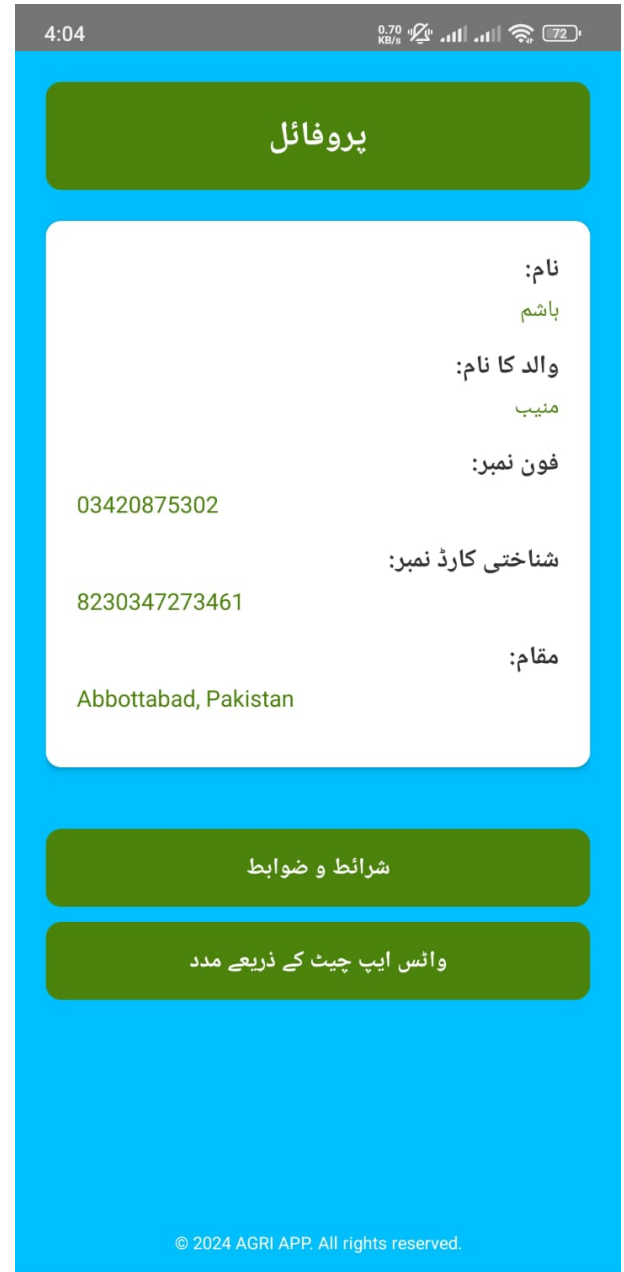


Figure 5.4 farmer dashboard and profile Screens

Soil Moisture Sensor



Temperature and humidity

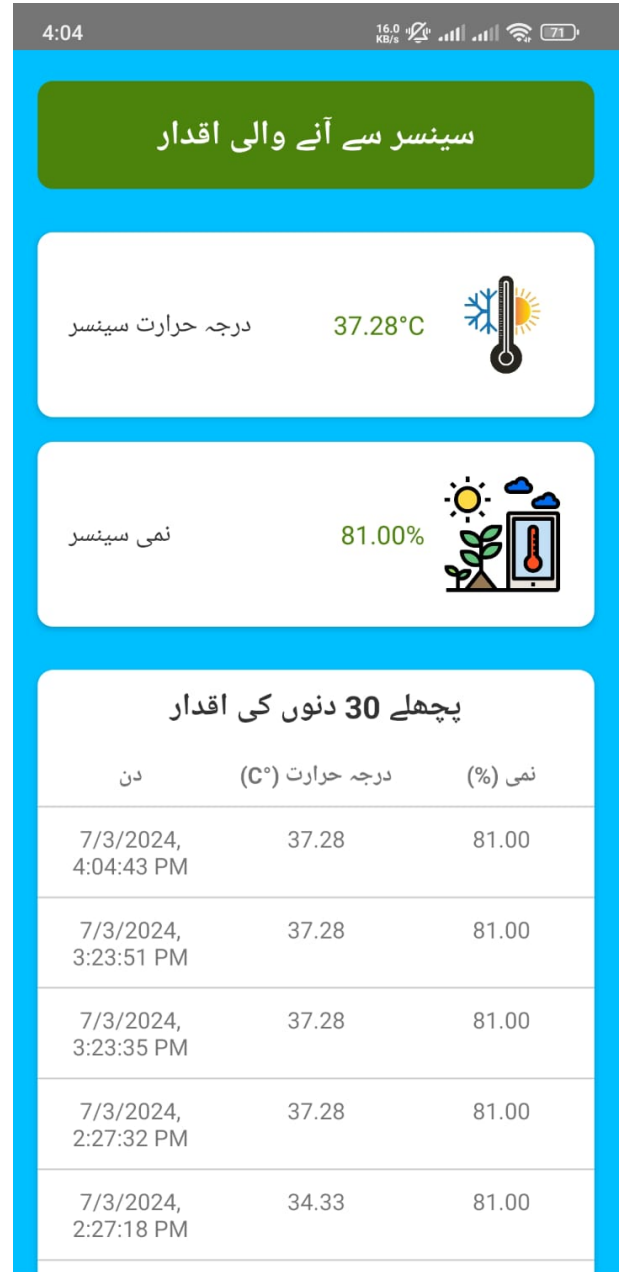


Figure 5.5 Soil Moisture and Real Time Monitoring Screens

NPK Sensor (Fertilizer need)



Storing Values of NPK

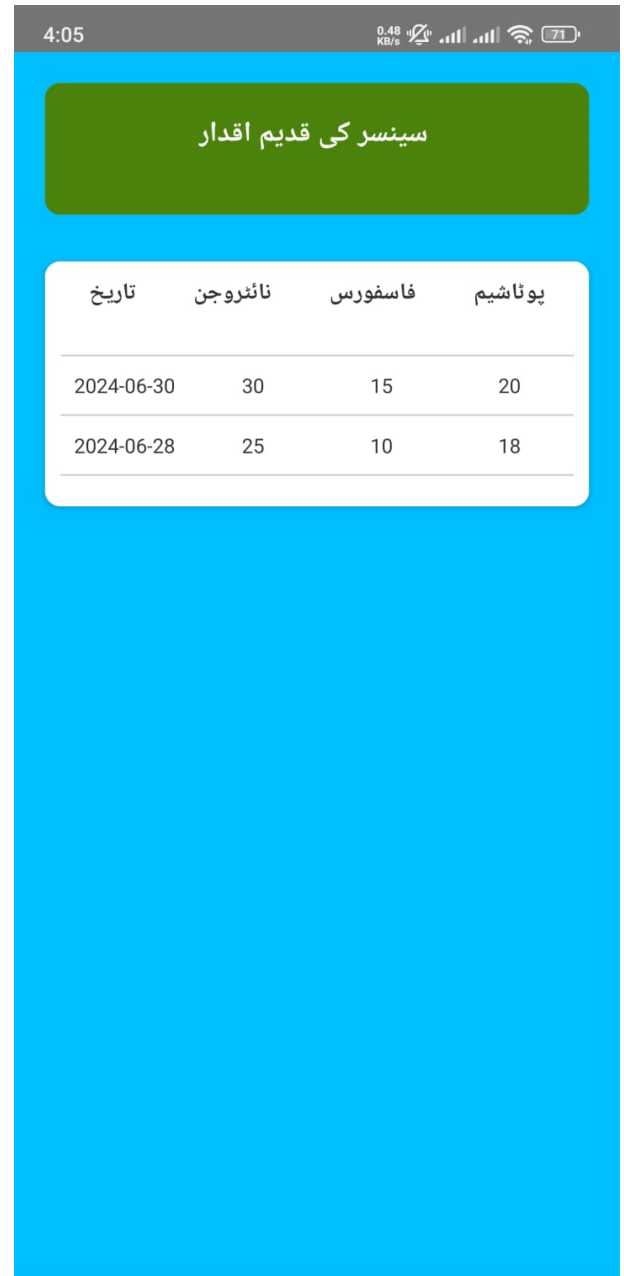


Figure 5.6 NPK Sensor and Values Screens

View and Rent a Machine



Figure 5.7 View Machines and see Reviews Screen

See Bookings and Book New

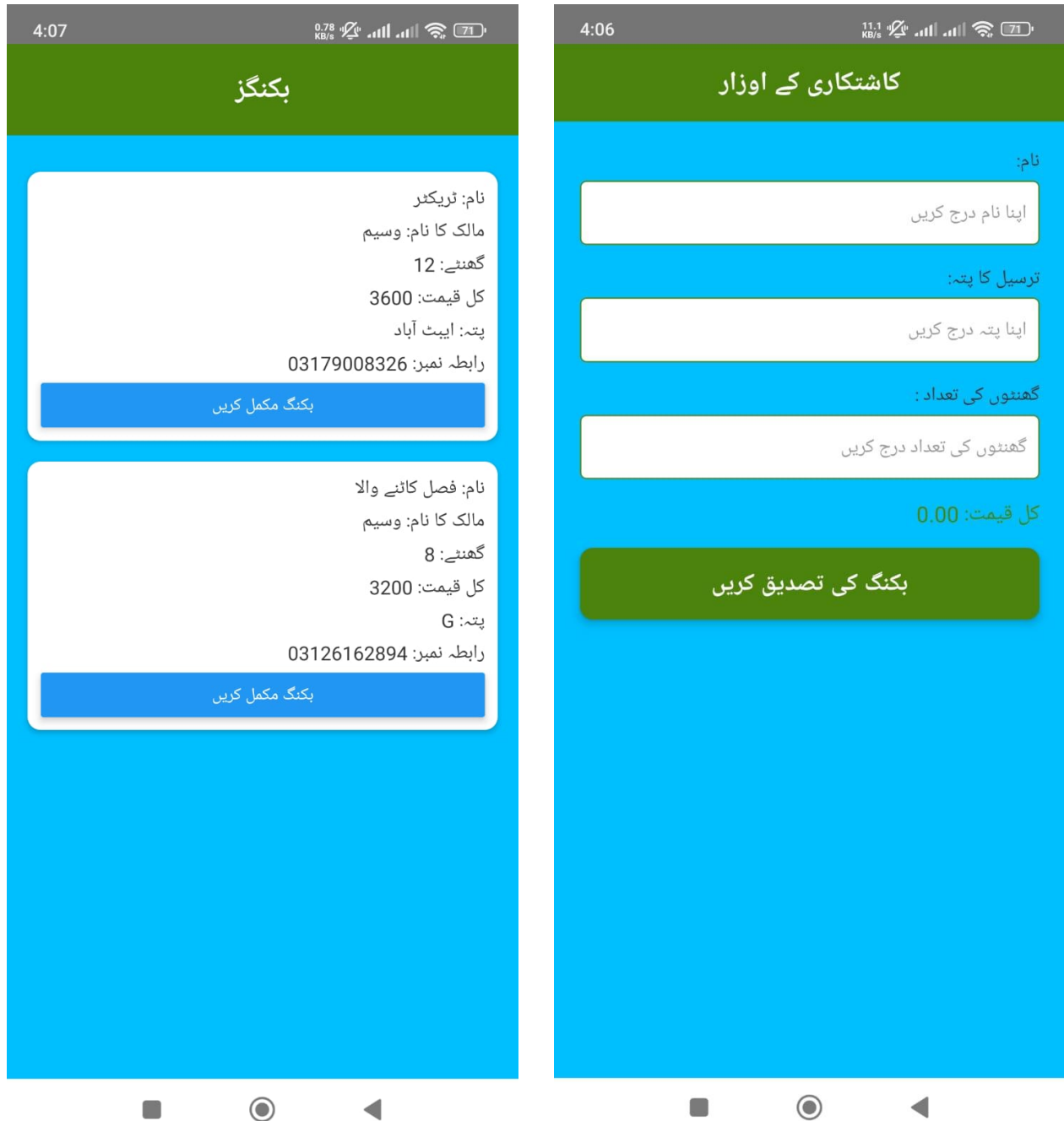


Figure 5.8 View booking and rent machine Screen

Loans and Insurances Options for Farmer



Figure 5.9 View loans information

Machine Owner Dashboard and Profile



Figure 5.10 Machine Owner Dashboard and Profile

Add, Delete, and update Machine

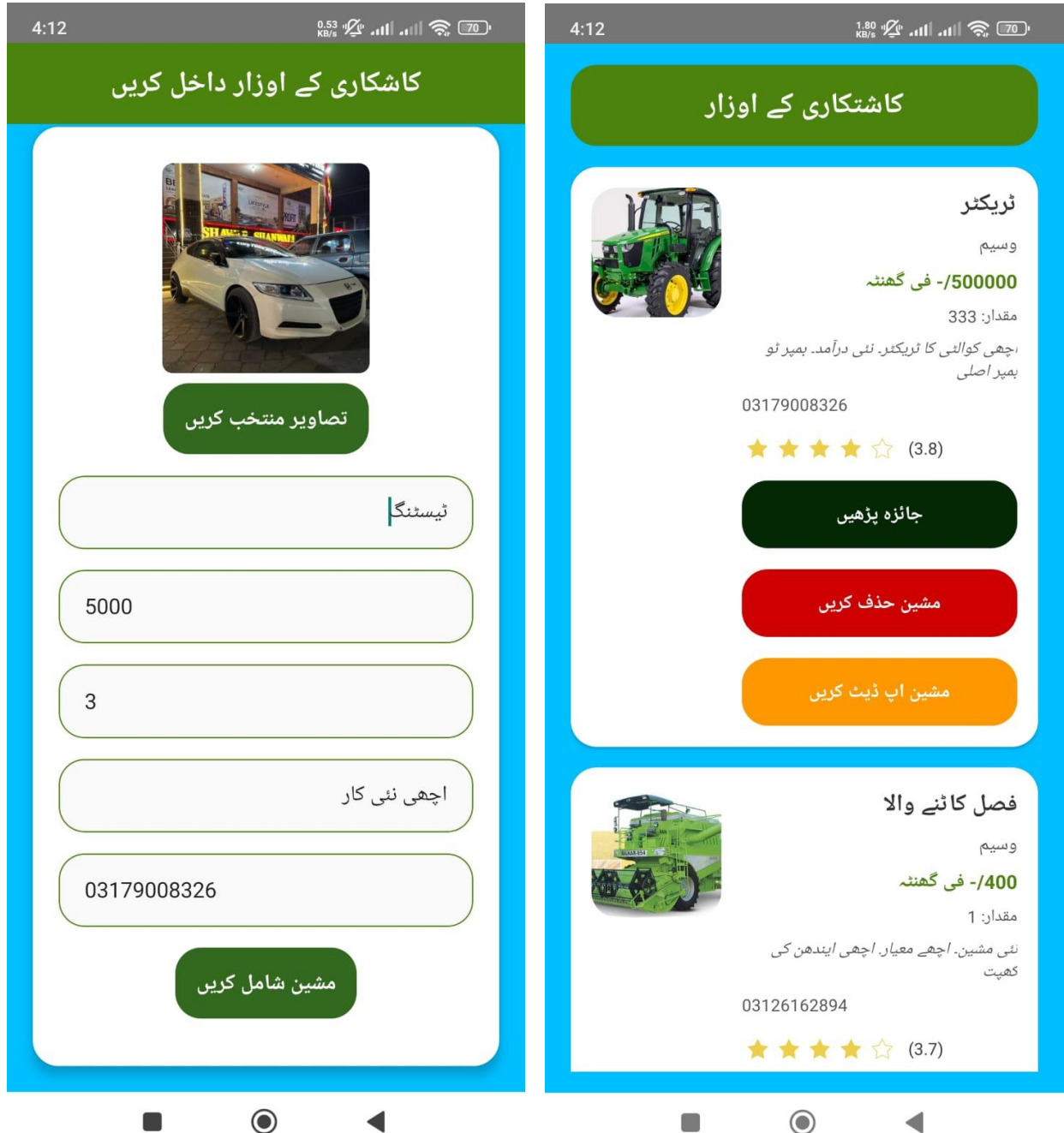


Figure 5.11 Add, update, and delete machine screens

Machine reviews and bookings

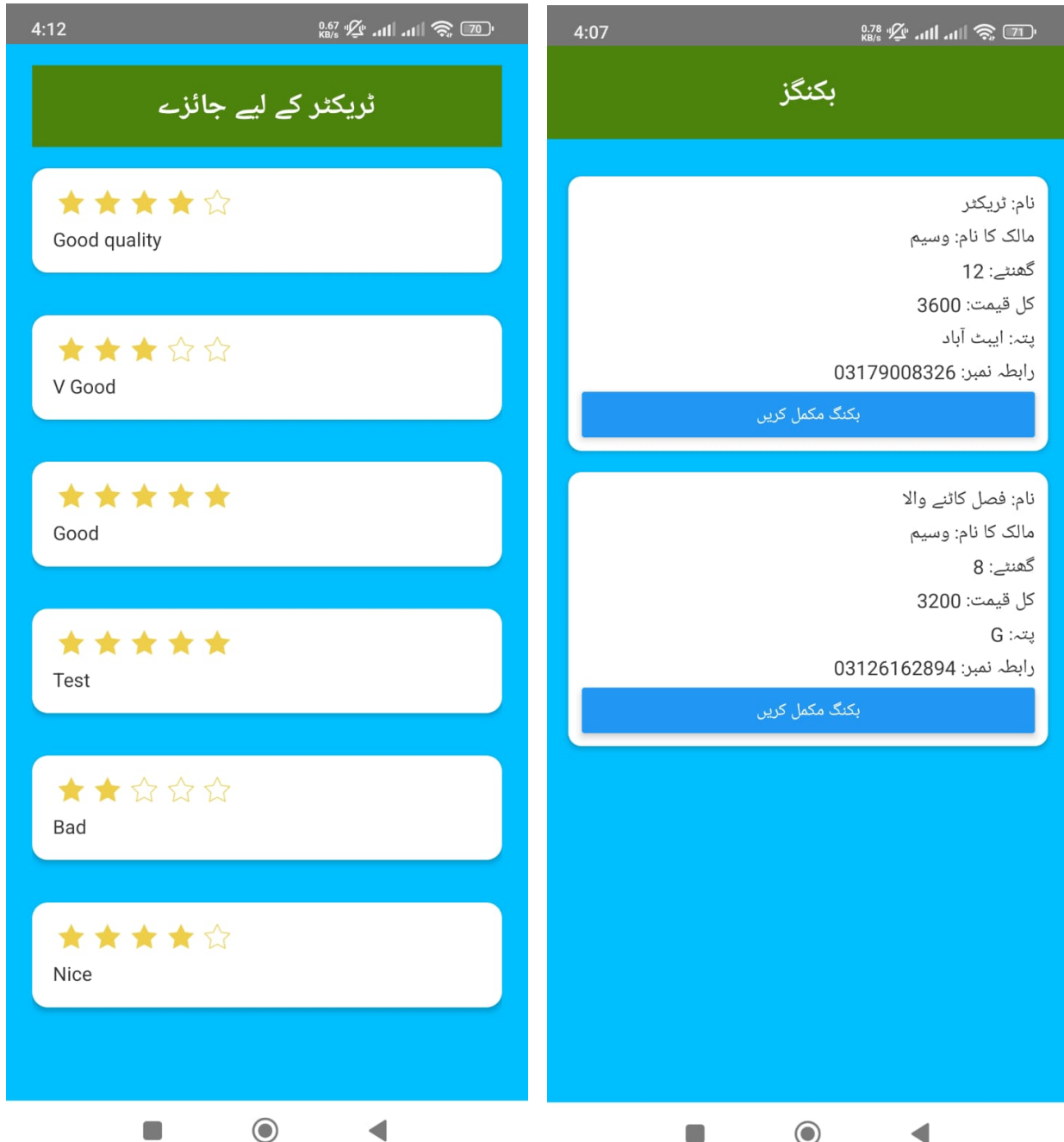


Figure 5.12 View rating and orders screen

6. Testing

Testing in AGRI APP involves validating the functionality, performance, and usability of the system. It includes various types of testing, such as functional testing to ensure that the website and its features work as intended, integration testing to verify seamless integration.

6.1. Manual Testing

Manual testing is a type of software testing where the testing activities are carried out manually by human testers without the use of automated testing tools. It involves the manual execution of test cases. Following are the types of manual testing that we performed:

6.1.1. System testing

After the Agri app has been constructed, system testing must be carried out to confirm that it performs as expected and conforms to the requirements that were originally established. The system's smooth and dependable operation is ensured by the testing phase's identification of any defects or issues that the user might not notice at first. All testing efforts (unit testing, functional testing, and integration testing) must be finished before releasing the application to users. This lessens the likelihood of introducing bugs or missing features that might negatively affect the user experience or get in the way of effective project management.

6.1.2. Unit Testing

Unit testing involves creating test cases for each unit of code and executing them independently. Following are the tables of Unit Testing:

- **Unit Testing 1: Sign Up**

Testing Objective: To validate that a user can successfully register for the system and create its profile.

Table 6.1. illustrates the unit testing of registration.

No	Test case/Test script	Attribute and value	Expected result	Result
1.	Verify that the user is successfully former and directed to the login page.	Name: Waseem Father Name: Tasawar Phone Number: 03126162894 CNIC: 3610489852905 Password: Waseem123	The Data is Saved in database to go to loginScreen.	Pass
2	Verify if the system correctly prompts the user to enter a value inthe “Name” field during sign-up when left empty.	Name field is left empty	System should prompt the user“Please fill out this field.”	Pass
3	Verify if the system correctly prompts the user to enter a value inthe “Password” field during sign-up when left empty.	Password field is left empty.	System should prompt the user“Please fill out this field.”	Pass
4	Verify if the system correctly prompts the user to enter a value inthe “Number” field during sign-up when left empty.	Number field is left empty.	System should prompt the user“Please fill out this field.”	Pass
5	Verify if the system correctly prompts the user to enter a value in the “CNIC” field during sign-up when left empty.	Number field is left empty.	Number field is left empty.	Pass
6	Verify if the system correctly prompts the user to enter a value in the “Father Name” field during sign-up when left empty.	Number field is left empty.	Number field is left empty.	Pass
7	Verify if the system correctly validates the length of the password	Password: 123456	System should prompt the	Pass

	during sign-up.		user “Password must be at least 8 Characters long.”	
--	-----------------	--	--	--

- **Unit Testing 2: Add Machinery**

Testing Objective: To validate if system is taking the all data required to add machinery.

Table 6.2: Upload Machinery Rental Owner Test Cases

No.	Test case/Test script	Attribute and value	Expected result	Result
1.	Verify if the system correctly validates the file format of the uploaded images.	Upload images in ‘.png’, ‘.jpg’ format.	System should prompt the user “Please upload a valid PNG or JPG image file.”	Pass
2.	Verify if the system allows the successful upload of images in PNG format.	Upload images in ‘.png’, ‘.jpg’ format.	Images should be uploaded	Pass
3.	Verify if the system allows entering the name of the machine	Enter the name of the machine.	System should accept the machine name without any errors.	Pass
4.	Verify if the system allows entering the contact number.	Enter a valid contact number.	System should accept the contact number without any errors.	Pass
5.	Verify if the system allows entering the detail of the machine.	Enter details of the machine.	System should accept the machine details without any errors.	Pass
6.	Verify if the system allows entering the quantity of the machine.	Enter quantity of the machine.	System should accept the quantity without any errors.	Pass
7.	Verify if the system allows entering the per hour price of the machine.	Enter per hour price of the machine.	System should accept the per hour price without any errors.	Pass
8	Verify if the system allows entering the quantity of the machine.	Enter Quantity of the machine.	System should accept the Quantity without any errors.	Pass

- **Unit Testing 3: Show Order Machinery**

Testing Objective: To validate if system is Show all data of place orders

Table 6.3: Fetch Machinery Orders Test Cases

No.	Test case/Test script	Attribute and value	Expected result	Result
1.	Verify if the system fetches the name of the purchaser correctly.	Fetch Name of Farmer from the database.	System should display the correct name of the Farmer.	Pass
2.	Verify if the system fetches the address of the purchaser correctly.	Fetch Address of Farmer from the database.	System should display the correct address of the Farmer.	Pass
3.	Verify if the system fetches the contact number of the Farmer correctly.	Fetch Contact Number from the database.	System should display the correct contact number of the Farmer.	Pass
4.	Verify if the system fetches the number of hours correctly.	Fetch Number of Hours from the database.	System should display the correct number of hours.	Pass
5.	Verify if the system fetches the price correctly.	Fetch Price from the database.	System should display the correct price.	Pass
6.	Verify if the system fetches the machine name correctly.	Fetch Machine Name from the database.	System should display the correct machine name.	Pass
7.	Verify if the system displays all order details correctly.	Fetch all order details from the database.	System should display the name, address, contact number, hours, price, and machine name correctly.	Pass

- **Unit Testing 4: Delete Machinery**

Testing Objective: To validate if system is Delete Machine

Table 6.4: Delete Machinery Test Cases

No.	Test case/Test script	Attribute and value	Expected result	Result
1.	Verify if the system shows a confirmation pop-up when the delete button is clicked.	Click on the Delete button.	System should display the correct name of the Former.	Pass
2.	Verify if the system deletes the machine from the interface when confirmed.	Click on Confirm in the pop-up.	System should show a pop-up with the message "Do you want to delete?".	Pass
3.	Verify if the system deletes the machine from the database when confirmed.	Click on Confirm in the pop-up.	System should remove the machine from the database.	Pass
4.	Verify if the system does not delete the machine when cancel is selected.	Click on Cancel in the pop-up.	System should not remove the machine from the interface or database.	Pass
5.	Verify if the pop-up disappears after confirming the delete action.	Click on Confirm in the pop-up.	Pop-up should disappear after the machine is deleted.	Pass
6.	Verify if the pop-up disappears after cancelling the delete action.	Click on Cancel in the pop-up.	Pop-up should disappear without deleting the machine.	Pass
7.	Verify if the system shows an error message if there is a failure in deleting the machine from the database.	Simulate a database deletion failure.	System should display an error message indicating the failure.	Pass

- **Unit Testing 5: Support Chat**

Testing Objective: To validate if system is Chat through WhatsApp

Table 6.5: Support Chat Test Cases

No.	Test case/Test script	Attribute and value	Expected result	Result
1.	Verify if the system shows the WhatsApp support option on the interface.	Navigate to the support section.	System should display an option to contact support via WhatsApp.	Pass
2.	Verify if the system opens WhatsApp with a pre-filled message template when the support option is clicked.	Click on the WhatsApp support option.	WhatsApp should open with the pre-configured support number and a pre-filled message template.	Pass
3.	Verify if the user can send a support message through WhatsApp successfully.	Send a message through WhatsApp.	Message should be sent to the support number successfully.	Pass
4.	Verify if the system logs the support request in the database after a message is sent.	Send a message through WhatsApp.	Support request should be logged in the system's database.	Pass

- **Unit Testing 6: Rental OOwner Logout**

Testing Objective: To validate if system is Logout the Owner

Table 6.6: Rental Owner Logout Test Cases

No.	Test case/Test script	Attribute and value	Expected result	Result
1.	Verify if the user can successfully log out of the system.	log out of the system. Click on Logout button	User should be logged out and redirected to the login page.	Pass

- **Unit Testing 6:** Sensor Data Integration

Testing Objective: To validate if the system correctly receives sensor data and updates the database.

Table 6.7: Sensor Data Integration Test Case

No.	Test case/Test script	Attribute and value	Expected result	Result
1.	Verify if the system receives temperature data from the sensor and updates the database.	Connect temperature sensor to NodeMCU.	System should receive temperature data and store it in the database.	Pass
2.	Verify if the system receives moisture data from the sensor and updates the database.	Connect moisture sensor to NodeMCU.	System should receive moisture data and store it in the database.	Pass
3.	Verify if the system receives humidity data from the sensor and updates the database.	Connect humidity sensor to NodeMCU.	System should receive humidity data and store it in the database.	Pass
4.	Verify if the system receives NPK (nitrogen, phosphorus, potassium) data from the sensor and updates the database.	Connect NPK sensor to NodeMCU.	System should receive NPK data and store it in the database.	Pass

- **Unit Testing 7:** Real-Time Monitoring - Display Temperature and Humidity

Testing Objective: To validate if the system correctly retrieves and displays real-time temperature and humidity values from the database on the user interface.

Table 6.8: Display Temperature and Humidity Test Case

No.	Test case/Test script	Attribute and value	Expected result	Result
1.	Verify if the system retrieves real-time temperature data from the database and displays it on the UI.	Access the "Real-Time Monitoring" module.	System should fetch the latest temperature value from the database and display it on the UI.	Pass
2.	Verify if the system retrieves real-time humidity data from the database and displays it on the UI.	Access the "Real-Time Monitoring" module.	System should fetch the latest humidity value from the database and display it on the UI.	

3.	Verify if the temperature value updates dynamically on the UI without manual refresh.	Monitor temperature changes in the database.	Temperature value displayed on the UI should update automatically as the database value changes.	
4.	Verify if the humidity value updates dynamically on the UI without manual refresh.	Monitor humidity changes in the database.	Humidity value displayed on the UI should update automatically as the database value changes.	
5.	Verify if the UI displays appropriate units (e.g., Celsius for temperature, percentage for humidity).	Check units displayed with temperature and humidity values.	Temperature should display in Celsius, and humidity should display in percentage on the UI.	

6.1.3. Functional Testing

Functional testing will take place after the unit testing. In this functional testing, the functionality of each of the modules is tested. This is to ensure that the system produced meets the specifications and requirements.

- **Functional Testing 1: User Login**

Objective: Verify that the user can login successfully into the system.

Table 6.9: User Login Test Cases

No.	Test case/Test script	Attribute and value	Expected result	Result
1.	Verify user login after click on the 'Login' button on login form with correct input data.	PhoneNumber: 03179008326 Password: 12345678	Successfully log onto the main page of the system as Hashim.	Pass
2.	Verify user login after click on the 'Login' button on login form with incorrect input data	Username: 12345 Password: a1b2	Gives Error: InvalidCredentials	Pass

- **Functional Testing 2: Sign Up**

Testing Objective: To validate that a user can successfully register for the system and create its profile. Table 6.1 illustrates the unit testing of registration.

Table 6.10: Rentel Owner Sign-Up Test Cases

No.	Test case/Test script	Attribute and value	Expected result	Result
1.	Verify that the user is successfully register and directed to the login page.	Name: Waseem Father Name: Tasawar Phone Number: 03126162 894 CNIC: 3610489852905 Password: waseem123	The Data is Saved in database to go to login Screen.	Pass
2.	Verify if the system correctly prompts the user to enter a value in the “Name” field during sign-up when left empty.	Name field is left empty	System should prompt the user “Please fill out this field.”	Pass
3.	Verify if the system correctly prompts the user to enter a value in the “Password” field during sign-up when left empty.	Password field is left empty.	System should prompt the user “Please fill out this field.”	Pass
4.	Verify if the system correctly prompts the user to enter a value in the “Number” field during sign-up when left empty.	Number field is left empty.	System should prompt the user “Please fill out this field.”	Pass
5.	Verify if the system correctly prompts the user to enter a value in the “CNIC” field during sign-up when left empty.	Number field is left empty.	Number field is left empty.	Pass

6.	Verify if the system correctly prompts the user to enter a value in the “Father Name” field during sign-up when left empty.	Number field is left empty.	Number field is left empty.	Pass
7.	Verify if the system correctly validates the length of the password during sign-up.	Password : 123456	System should prompt the user “Password must be at least 8 Characters long.”	Pass

- **Functional Testing 3: Machine Rental Process**

Testing Objective: To validate if the system allows a farmer to rent a machine only if the quantity is greater than 0.

Table 6.11: Machine Rental Process Test Case

No.	Test case/Test script	Attribute and value	Expected result	Result
1.	Verify if the system allows a farmer to enter their name, address, and rental hours.	Click on the ‘Download’ Icon and choose the format.	System should accept the name, address, and hours without errors.	Pass
2.	Verify if the system prevents renting a machine when quantity is 0.	Try to rent a machine with quantity 0.	System should display an error message and prevent the rental.	Pass
3.	Verify if the system allows renting a machine when quantity is greater than 0.	Rent a machine with quantity greater than 0.	System should complete the rental process successfully.	Pass
4.	Verify if the system updates the machine quantity correctly after rental.	Rent a machine and check the quantity.	Quantity of the machine should decrease by the rented amount	Pass
5.	Verify if the system displays a confirmation message after successful rental.	Complete the rental process.	System should display a confirmation message indicating successful rental.	Pass

- **Functional Testing 4:** Review Process

Testing Objective: To validate if the system allows a farmer to give a rating and comment after finishing an order.

Table 6.12: Review Process Test Case

No.	Test case/Test script	Attribute and value	Expected result	Result
1.	Verify if the system allows a farmer to give a rating after finishing an order.	Complete an order and access the review section.	System should allow the farmer to select a rating.	Pass
2.	Verify if the system allows a farmer to give a comment after finishing an order.	Complete an order and access the review section.	System should allow the farmer to enter a comment.	Pass
3.	Verify if the system successfully saves the rating given by the farmer.	Submit a rating for the finished order.	System should save the rating to the database and confirm the submission.	Pass
4.	Verify if the system successfully saves the comment given by the farmer.	Submit a comment for the finished order.	System should save the comment to the database and confirm the submission.	Pass
5.	Verify if the system displays an error message if the rating is not provided.	Try to submit a review without selecting a rating.	System should display an error message prompting the user to select a rating.	Pass
6.	Verify if the system displays a confirmation message after a successful review submission.	Submit a rating and comment for the finished order.	System should display a confirmation message indicating successful review submission.	Pass

6.1.4. Integration Testing

Integration testing involves testing the interfaces and interactions between various software modules, subsystems, or external systems to ensure that they work correctly together. This Section shows the Integration Testing:

- **Integration Testing 1:** Check Previous Booking
Testing Objective: To validate if system has previously Booking of the user.

Table 6.13: Check Previous Designs Test Cases

No.	Test case/Test script	Attribute and value	Expected result	Result
1.	Sign in	Enter Valid Credentials that were used to Sign Up.	Login successful and direct to the home page.	Pass
2.	Verify that the user can view, rented Items	Check the Booking, which have all bookings machine	All the rented machine in it	Pass

- **Integration Testing 2:** NPK Sensor Data Handling
Testing Objective: To validate if the system correctly receives NPK sensor data, stores previous values in the database, and displays them on the UI.

Table 6.14. NPK Sensor Data Handling Test Cases

No.	Test case/Test script	Attribute and value	Expected result	Result
1.	Verify if the system receives NPK data from the sensor and updates the database.	Connect NPK sensor to NodeMCU and send	System should receive NPK data and store it in the database.	Pass
2.	Verify if the system stores previous NPK values in the database.	Connect NPK sensor and send multiple data points over time.	The system should store all previous NPK values in the database.	Pass

3.	Verify if the system retrieves and displays the latest NPK value on the UI.	Access the real-time monitoring module.	System should display the latest NPK value on the UI.	Pass
4.	Verify if the system retrieves and displays previous NPK values on the UI.	Access the history or data log section.	System should display a list or chart of previous NPK values on the UI.	Pass
5.	Verify if the UI updates dynamically with new NPK values without manual refresh.	Monitor NPK value changes and observe UI updates.	NPK value displayed on the UI should update automatically as new data is received.	Pass
6.	Verify if the system handles missing or null NPK data gracefully.	Simulate a scenario where NPK sensor sends null or no data.	System should handle missing data gracefully without crashing.	Pass

7. Conclusion and Future Work

In conclusion, implementing an Agri app for farmer Help can bring significant benefits to the Agriculture industry. In future, we can enhance the user experience by adding more customized features, more options to guide farmer in a better way. In future, going to add cross-platform compatibility and collaborations with international clients.

7.1. Conclusion

In conclusion, the agriculture-focused mobile application we developed stands as a significant milestone in modernizing and streamlining agricultural operations. By addressing the critical needs of both farmers and machine owners, the app has successfully created a platform that promotes efficiency, resource-sharing, and sustainable farming practices. The primary goal of the application is to facilitate the renting and management of agricultural machinery, which traditionally has been a cumbersome and time-consuming process. Through a user-friendly interface and seamless integration with Firestore for real-time data management, our app simplifies these tasks, ensuring that farmers can easily access the tools they need without unnecessary delays.

The integration of sensor technology within the app is another noteworthy achievement. By incorporating sensors that monitor water and fertilizer needs, as well

as real-time temperature and moisture levels, the app provides farmers with critical data that can significantly impact their crop management decisions. This real-time monitoring allows for precise irrigation and fertilization, ensuring that crops receive the optimal amount of resources at the right times. Consequently, this leads to healthier crops, higher yields, and more efficient use of resources, which are essential for sustainable farming.

Additionally, the app's emphasis on local language support ensures that it is accessible to a diverse user base, including those who may not be fluent in English. This inclusivity is crucial for broader adoption and ensures that all farmers, regardless of their linguistic background, can benefit from the app's features. The ability to translate key elements into Urdu and provide error messages in the local language enhances the user experience, making the app more intuitive and easier to use.

The app also fosters a community-oriented approach by enabling farmers and machine owners to share resources. This resource-sharing not only reduces costs but also maximizes the utilization of machinery, which is often underused in individual farms. By facilitating this collaboration, the app contributes to a more sustainable and economically viable farming ecosystem.

Moreover, the app's real-time data and analytics empower users to make informed decisions. Whether it's deciding the best time to water crops, determining the appropriate amount of fertilizer, or managing machinery efficiently, the app provides the necessary insights to optimize these processes. This data-driven approach is a significant step forward from traditional farming methods, which often rely on intuition and experience rather than precise data.

The feedback mechanism incorporated into the app also allows for continuous improvement. Users can rate and review machinery, providing valuable insights to other farmers and helping machine owners maintain high standards of service. This transparency fosters trust and accountability within the agricultural community, further enhancing the app's value proposition.

7.2. Future Work

Looking ahead, there are several exciting areas where the application can be expanded and improved:

Advanced Analytics and Predictive Maintenance: Implementing advanced data analytics and machine learning algorithms to predict machinery maintenance needs and optimize usage. This can help prevent breakdowns, reduce maintenance costs, and extend the lifespan of the equipment. By analyzing usage patterns and performance data, the app can provide timely alerts and recommendations for preventive maintenance.

Enhanced Sensor Integration: Expanding the sensor capabilities to include more parameters such as soil pH, nutrient levels, and pest detection. This will provide farmers with a more comprehensive understanding of their fields and help in making

precise interventions. Advanced sensors can detect early signs of nutrient deficiencies or pest infestations, allowing for timely and targeted treatments.

Integration with IoT Devices: Enhancing the app with Internet of Things (IoT) capabilities to provide real-time monitoring and control of machinery, soil conditions, and weather forecasts. This can further improve efficiency and decision-making. IoT devices can automate irrigation systems based on real-time soil moisture data, ensuring optimal water usage and reducing wastage.

Financial Services Integration: Introducing features for financial management, such as loan applications, insurance, and payment gateways. This would provide users with comprehensive financial support and services directly through the app. Access to financial services can help farmers invest in better machinery and inputs, improving their productivity and profitability.

Community and Knowledge Sharing Platform: Creating a community forum for farmers and machine owners to share experiences, best practices, and solutions to common problems. Additionally, integrating educational resources and training modules can help users stay updated with the latest agricultural techniques and technologies. This platform can facilitate knowledge exchange and foster a collaborative learning environment.

Expansion to Other Regions: Scaling the app to support more languages and cater to the agricultural needs of different regions and countries. This can help in reaching a broader audience and addressing diverse agricultural challenges. By understanding regional farming practices and challenges, the app can provide tailored solutions and recommendations.

Enhanced Security and Data Privacy: Continuously improving security measures to protect user data and ensuring compliance with global data privacy regulations. This will build trust and encourage more users to adopt the app. Implementing robust encryption and secure authentication mechanisms can safeguard user information from unauthorized access.

Offline Functionality: Developing offline capabilities to ensure that users in remote areas with limited internet connectivity can still access essential features and data. Offline data synchronization can enable users to record and access critical information even without an active internet connection.

8. References

1. Chen, J., & Yu, Y. (2019). *Smart Farming Technologies*. Springer.
2. Zhang, Q. (2015). *Precision Agriculture Technology for Crop Farming*. CRC Press.
3. Lee, I., & Siau, K. (2017). A Review of Smart Farming Technologies in Precision Agriculture. *Journal of Advanced Agricultural Technologies*, 4(2), 138-144.
4. Wolfert, S., Ge, L., Verdouw, C., & Bogaardt, M. J. (2017). Big Data in Smart Farming – A review. *Agricultural Systems*, 153, 69-80.
5. Kamilaris, A., Kartakoullis, A., & Prenafeta-Boldú, F. X. (2017). A Review on the Practice of Big Data Analysis in Agriculture. *Computers and Electronics in Agriculture*, 143, 23-37.
6. Grogan, T. (2019). *Blockchain for Agriculture: Realizing the Potential*. CRC Press.
7. Kumar, V. (2016). *Agriculture 4.0: Agricultural Internet of Things and Decision Support for Smart Farming*. Springer.
8. Ahmed, N., De, D., & Hussain, I. (2016). Internet of Things (IoT) for Smart Precision Agriculture and Farming in Rural Areas. *IEEE Internet of Things Journal*, 3(5), 778-786.
9. Savita, K., & Singh, V. (2018). *Sensors and Systems for Agriculture*. Elsevier.
10. Balafoutis, A., Fountas, S., Cavalaris, C., Vangeyte, J., Van der Wal, T., Soto, I., ... & Evert, F. (2017). Precision Agriculture Technologies Positively Contributing to GHG Emissions Mitigation, Farm Productivity and Economics. *Sustainability*, 9(8), 1339.
11. Carson, D. J. (2015). *ICT and Agriculture*. Springer.
12. Chatzimichail, E., Theodorou, C., & Lepouras, G. (2018). Blockchain Technology: Applications in the Internet of Things. *IEEE Communications Surveys & Tutorials*, 20(4), 3173-3200.
13. Jayaraman, P. P., Yang, Z., & Yavari, A. (2017). Internet of Things Platform for Smart Farming: Experiences and Lessons Learnt. *Sensors*, 17(12), 2772.
14. Atzori, L., Iera, A., & Morabito, G. (2010). The Internet of Things: A survey. *Computer Networks*, 54(15), 2787-2805.

ORIGINALITY REPORT

16%

SIMILARITY INDEX

13%

INTERNET SOURCES

1%

PUBLICATIONS

14%STUDENT PAPERS

PRIMARY SOURCES

1**Submitted to Higher Education Commission
Pakistan**

Student Paper

6%

2**vehari.comsats.edu.pk**

Internet Source

1%

3**Submitted to Edith Cowan University**

Student Paper

1%

4**Submitted to University of Malaya**

Student Paper

1%

5**Submitted to Birkbeck College**

Student Paper

1%

6**Submitted to Kookmin University**

Student Paper

1%

7**Submitted to Morgan Park High School**

Student Paper

<1%

8**Submitted to Leeds Beckett University**

Student Paper

<1%

9**Submitted to Universiti Teknologi Malaysia**

Student Paper

<1%

10	dokumen.pub Internet Source	<1%
11	Submitted to UOW Malaysia KDU University College Sdn. Bhd Student Paper	<1%
12	123dok.com Internet Source	<1%
13	acseusa.org Internet Source	<1%
14	files.library.northwestern.edu Internet Source	<1%
15	Submitted to Multimedia University Student Paper	<1%
16	Submitted to Taylor's Education Group Student Paper	<1%
17	Submitted to Canterbury Christ Church University Student Paper	<1%
18	Submitted to Florida International University Student Paper	<1%
19	Submitted to NCC Education Student Paper	<1%
20	Submitted to <u>University</u> of Sunderland Student Paper	<1%